

Reasoning-aware Adaptive Routing for Hybrid GraphRAG

Cong Tran¹

Nguyen Dinh Thanh²

Pham Van Hai²

¹Posts and Telecommunications Institute of Technology (PTIT), Hanoi, Vietnam

²School of Information Technology and Communication, Hanoi University of Science and Technology, Hanoi, Vietnam
`thanh.nd225670@sis.hust.edu.vn`

Abstract

In Vietnamese legal question answering, the appropriate retrieval strategy varies with the query: simple questions require direct textual lookup, relational questions demand reasoning over legal structures such as amendment, replacement, and cross-document reference, while ambiguous questions risk grounding answers in the wrong legal target. We propose a Hybrid GraphRAG system with a two-stage Reasoning-aware Adaptive Router that selects among dense retrieval, graph traversal, hybrid reasoning, and clarification. A lightweight XGBoost first stage handles clear cases, while a selective LLM verifier is invoked only for uncertain, ambiguous, or relation-heavy queries; a deterministic history resolver decides whether a conversational reference can be resolved before retrieval. The system operates over a dense index of legal text chunks and a large Vietnamese legal knowledge graph built from the official electronic legal codification (*Bộ Pháp điển điện tử*). Through experiments on a strict routing benchmark, an ambiguity diagnostic, and a conversation stress test, we demonstrate (a) that the two-stage router improves end-to-end answer quality over fixed-strategy baselines and even over an oracle with

perfect routing labels, indicating that the second stage acts as a Chain-of-Thought step; (b) that the first-stage classifier routes reliably at millisecond latency; (c) that clarification is detected with perfect precision and zero false positives; and (d) that selective second-stage invocation yields a favourable quality-latency trade-off.

Keywords: Retrieval-Augmented Generation, GraphRAG, Hybrid RAG, Adaptive Routing, Vietnamese Legal QA, Knowledge Graphs, Multi-hop Reasoning, Ambiguity Detection.

1 Introduction

1.1 Background and Motivation

Retrieval-Augmented Generation (RAG) has become a standard approach for grounding language-model outputs in external evidence, improving factual reliability without retraining the underlying model [14, 9]. In its most common form, RAG relies on dense passage retrieval, which embeds a query and a corpus into a shared vector space and returns the most semantically similar text chunks [13]. This paradigm is effective for factoid questions whose answer is stated directly in a single passage. In the original formulation, however, the corpus is treated as a flat collection of independent chunks, and retrieval is

reduced to nearest-neighbor search over isolated text spans [13, 27].

In practice, this assumption is overly simplistic for many real-world question-answering settings. When the answer depends on relations among entities, on document structure, or on chains of evidence spanning multiple documents, dense retrieval cannot follow those connections because the relations are not represented in the vector space [27]. Graph-based retrieval was introduced precisely to address this gap: by organizing the corpus as a knowledge graph, retrieval can traverse explicit, typed relationships rather than relying solely on surface similarity [6, 29, 12]. Yet graph traversal is more expensive than dense lookup, and applying it to every query—including simple ones whose answer lies in a single article—incurs unnecessary latency and computational cost. Vietnamese legal question answering exposes this tension sharply: the same system must serve both simple article-level definitions and multi-document questions involving amendment, replacement, and cross-document reference [2, 31, 24], while Vietnamese-specific word segmentation and multi-syllable named entities further complicate legal reference matching [18, 28].

When confronted with such heterogeneous queries, a naturally arising question is how to decide *which* retrieval strategy a given query requires, *before* any evidence is retrieved. Existing Hybrid RAG systems sidestep this question by applying dense and graph retrieval indiscriminately, paying the cost of both regardless of query complexity [6, ?, ?]. Recent adaptive-routing methods show that selecting a strategy by query complexity improves both quality and efficiency [10, 4, 19], but they route among *model sizes* or *retrieval depths* rather than among distinct *evidence mechanisms*; the

decision of whether a query needs relational traversal at all is left unaddressed. A separate and safety-critical gap concerns ambiguity. When a user asks “*Văn bản đó còn hiệu lực không?*” (“*Is that document still in effect?*”) without a usable referent in the conversation, forcing retrieval grounds the answer in an arbitrary legal document and returns a confidently wrong answer [17, 7]. No prior system simultaneously decides among dense retrieval, graph traversal, hybrid reasoning, and clarification for Vietnamese legal QA.

Motivated by the observation that the appropriate retrieval strategy is a property of the query itself—and can be predicted cheaply from the query and its conversational context—we propose a two-stage Reasoning-aware Adaptive Router that selects one of four evidence actions before retrieval. A lightweight XGBoost classifier [5] handles clear cases at low cost; a selective LLM verifier is invoked only for uncertain, ambiguous, or relation-heavy queries, following the cost-aware cascade principle [4, 19]. A deterministic **HistoryResolver** precedes both stages to distinguish resolved, irrelevant, and conflicting conversation references, so that clarification is triggered only when the legal target is genuinely underspecified.

1.2 Main Contributions

To address the heterogeneity of legal queries described above, we adopt a design principle in which the retrieval strategy is selected by a two-stage router operating upstream of retrieval, rather than by running all strategies and merging their outputs. The first stage performs a fast, feature-based classification of the query into a retrieval mechanism using an XGBoost model trained on routing labels derived from evidence structure. Because this stage is lightweight, it

can serve as a real-time gateway that resolves the majority of clear cases without invoking a large model. The second stage is an LLM verifier that is triggered selectively, only when the first-stage decision is uncertain, when the query carries strong relational signals, or when the conversational referent cannot be resolved. This selective design keeps the expected per-query cost low while reserving explicit reasoning for the cases that need it. Crucially, the second stage does more than correct routing decisions: by reasoning explicitly over query intent and legal constraints before answering, it also functions as a Chain-of-Thought step that improves the quality of the generated answer. Clarification is integrated as a first-class routing action so that, when no retrieval strategy can safely ground the answer, the system asks the user rather than guessing.

Our contributions are four-fold:

- We formulate reasoning-aware retrieval routing for Vietnamese legal QA as a four-class decision—dense retrieval, graph traversal, hybrid reasoning, and clarification—made before evidence retrieval, unifying mechanism selection and ambiguity handling in a single framework [10, 17].
- We design a two-stage router in which a lightweight XGBoost stage handles clear cases and a selective LLM verifier is invoked only for uncertain, ambiguous, or relation-heavy queries, and we show that this verifier acts as an explicit Chain-of-Thought mechanism: the Two-stage Hybrid ($F1 = 0.661$) outperforms an Oracle Router with perfect routing labels ($F1 = 0.606$), demonstrating that Stage 2 reasoning improves answer generation beyond correct route selection

alone [4].

- We introduce a deterministic **HistoryResolver** that classifies a conversational referent as resolved, irrelevant, or conflicting, enabling the router to unblock retrieval when a referent is available and to force clarification when it is absent [7].
- Through experiments on a strict 600-query benchmark, a 234-query ambiguity benchmark, and a 160-query conversation stress test, we validate (a) end-to-end answer quality against fixed-strategy and oracle baselines, (b) routing accuracy against rule-based and learned baselines, (c) clarification behavior, and (d) the cost-quality trade-off of selective verification.

To the best of our knowledge, this is the first system to treat clarification as a first-class routing action alongside dense, graph, and hybrid retrieval in a Hybrid GraphRAG pipeline for Vietnamese legal question answering [24, 31].

1.3 Organization

Section 2 reviews related work. Section 3 defines the routing problem, dataset, and pipeline. Section 4 details the routing framework. Sections 5 and 6 present experimental setup and results. Section 7 discusses limitations before Section 8 concludes.

2 Related Work

Retrieval-Augmented Generation. RAG combines parametric knowledge from neural language models with non-parametric knowledge retrieved from an external corpus [14, 9]. Dense retrieval is effective for open-domain factoid QA but degrades on multi-hop evidence chains [13, 27]. A line of advanced variants

improves retrieval control—Self-RAG through self-reflection [1], FLARE through active retrieval [11], RAPTOR through hierarchical summaries [22], and CRAG through corrective re-retrieval [30]. These methods refine *how* dense retrieval is performed, but they operate within a single evidence mechanism and do not decide whether a query requires relational traversal or clarification.

Graph Retrieval and GraphRAG. Graph-based retrieval lets evidence follow explicit relationships [25, 29]. GraphRAG builds entity graphs for query-focused summarization [6]; HippoRAG introduces a neurobiologically inspired graph memory for associative retrieval [12]; LightRAG uses dual-level retrieval over entity and community knowledge [?]. For legal QA, graph retrieval is especially relevant because legal documents encode structured relations such as amendment, replacement, and jurisdiction [24]. However, these systems assume graph traversal is always applied; none decides *when* traversal is warranted versus when a dense lookup would suffice at lower cost.

Adaptive Routing and Cost-aware Inference. Adaptive-RAG selects a retrieval strategy according to question complexity [10], while FrugalGPT and RouteLLM route queries across models of different cost [4, 19]. These works establish the cost-aware cascade principle that motivates our design. Their routing target, however, is the *model* or *retrieval depth*; they do not route among qualitatively different evidence mechanisms, and they do not treat clarification as a routable action.

Ambiguity Handling. CANARD studies question rewriting for context-dependent questions [7], and AmbigQA shows that open-domain questions often admit multiple valid interpretations [17]. These studies treat ambiguity resolu-

tion as a stand-alone task applied before or after retrieval, rather than as one possible outcome of a unified routing decision.

Vietnamese NLP and Legal QA. Vn-CoreNLP provides Vietnamese word segmentation and NER [28], and PhoBERT provides pre-trained Vietnamese representations [18]. Legal NLP more broadly requires reasoning over regulatory relations [2, 31, 8, ?]. This body of work supplies the linguistic and legal-domain foundations our system builds on, but does not address adaptive retrieval routing.

Discussion. Despite substantial progress along each of these lines, no prior system decides among dense retrieval, graph traversal, hybrid reasoning, and clarification *before* retrieval in a single framework. Advanced-RAG methods improve a single mechanism; GraphRAG methods assume traversal is always needed; cost-aware routers select models rather than evidence mechanisms; and ambiguity-handling methods treat clarification in isolation. Our work differs in placing an explicit, reasoning-aware routing decision at the center of the pipeline, and in treating clarification as a first-class route backed by deterministic conversational context resolution. Table 1 summarizes these research lines and the specific limitation that each leaves open for Vietnamese legal QA.

Table 1: Summary of related research lines.

Line (role)	Repr. work	Limitation for this work
Dense RAG [14, 13] (dense)	RAG, DPR, REALM	Text retrieval lacks explicit legal relations.
Advanced RAG [1, 11, 22] (control)	Self-RAG, FLARE, RAPTOR	Improves control, not four-way legal routing.
Graph RAG [6, 29, 12] (graph)	GraphRAG, KGP, HippoRAG	Does not decide <i>when</i> graph is needed.
Routing [10, 4, 19] (router)	Adaptive-RAG, Frugal-GPT	Routes model sizes, not evidence mechanisms.
Ambiguity QA [17, 7] (clarify)	AmbigQA, CANARD	Clarification not integrated as a routing action.
VN Legal NLP [18, 2] (features)	PhoBERT, LegalBERT	Provides NLP foundations, not adaptive routing.
Ours (4-way)	This work	Routes by reasoning, ambiguity, and cost.

3 Methodology

3.1 Problem Definition

Let $\mathcal{C} = \{D_1, \dots, D_N\}$ denote the Vietnamese legal corpus. From $\mathcal{C}$, the system builds two complementary knowledge sources: a dense vector index $\mathcal{I}_v = \{(c_j, \mathbf{e}_j)\}_{j=1}^M$ of M legal text chunks, and a legal knowledge graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ where a node $v \in \mathcal{V}$ represents a legal document, article, semantic concept, agency, or other entity, and an edge $(u, r, v) \in \mathcal{E}$ represents a typed relation such as HAS_ARTICLE, MENTIONS, AMENDS, or REPLACES.

At inference time, the input is the current query q_t at turn $t \geq 1$, together with optional conversation history $H = \{(q_i, a_i)\}_{i=1}^{t-1}$. When $t = 1$, $H = \emptyset$. The router selects one action from the route set $\mathcal{D} = \{d_v, d_g, d_h, d_c\}$, where d_v , d_g , d_h , d_c denote **dense_retrieval**, **graph_traversal**, **hybrid_reasoning**, and **clarify**, respectively.

The routing function is

$$\rho : (q_t, H) \rightarrow d^*, \quad d^* \in \mathcal{D}. \quad (1)$$

For retrieval routes, the evidence set is

$$E_{d^*} = \begin{cases} \text{TopK}(q_t, \mathcal{I}_v, k), & d^* = d_v, \\ \text{Traverse}(q_t, \mathcal{G}, h), & d^* = d_g, \\ E_v \cup E_g, & d^* = d_h, \end{cases} \quad (2)$$

where k is the dense retrieval depth and h is the traversal depth. The generator produces $a_t = \text{Generate}(q_t, H, E_{d^*})$. If $d^* = d_c$, no evidence is retrieved; instead the system returns a clarification question $c_t = \text{Clarify}(q_t, H)$.

3.2 Dataset

The evaluation uses a self-constructed Vietnamese legal QA routing dataset derived from a single source: the Vietnamese electronic legal codification system (*Bộ Pháp điển điện tử*), flattened into article-level records. QA pairs are linked to supporting articles through an **article_key** (format `DOC::ARTICLE_ID`) and assigned routing labels based on evidence structure: **dense_retrieval** for single-article questions, **graph_traversal** for multi-article reasoning within one document, and **hybrid_reasoning** for evidence spanning multiple documents. Low-quality samples are removed by deduplication, length filtering, and placeholder-answer filtering.

Table 2 summarizes the dataset splits and Figure 1 shows the route distribution. The dataset remains skewed toward dense retrieval, which accounts for half of the samples (50%), motivating Macro-F1 reporting alongside accuracy. The strict split is used for retrieval-route evaluation; the **clarify** route is evaluated separately on a 234-query ambiguity benchmark (156 ambiguous, 78 answerable controls) and a 160-query conversation stress test.

Table 2: Vietnamese legal QA routing dataset split.

Split	Dense	Graph	Hybrid	Total
Train	400	200	200	800
Dev	50	25	25	100
Test	300	150	150	600
Total	750	375	375	1,500

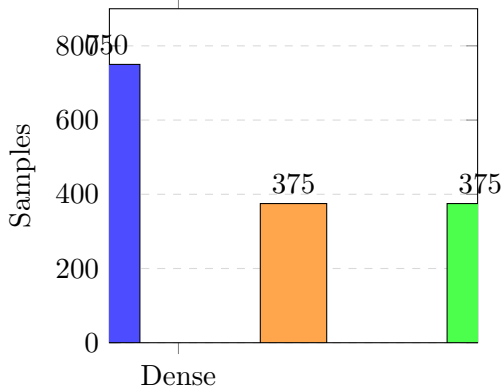


Figure 1: Route-label distribution. Dense retrieval (50%) accounts for half of the dataset, motivating Macro-F1 reporting alongside accuracy.

3.3 System Overview

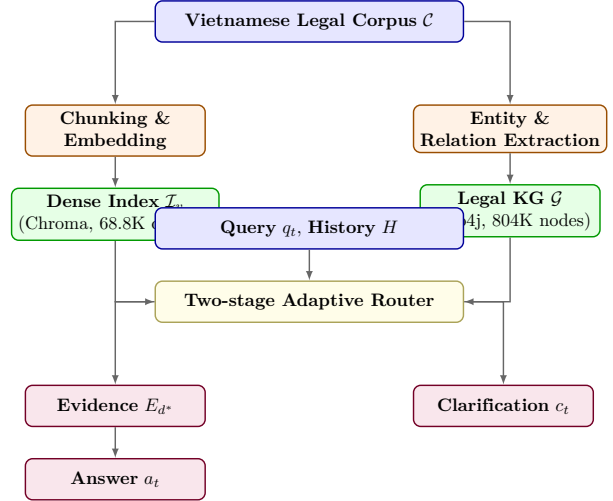


Figure 2: High-level data flow. The Vietnamese legal corpus $\mathcal{C}$ is processed offline into a dense vector index $\mathcal{I}_v$ (ChromaDB) and a legal knowledge graph $\mathcal{G}$ (Neo4j). At query time, the two-stage adaptive router selects evidence from $\mathcal{I}_v$, $\mathcal{G}$, or both, or returns a clarification question c_t .

Figure 2 shows the overall data flow. In the offline phase, $\mathcal{C}$ is indexed once into $\mathcal{I}_v$ and $\mathcal{G}$. In the online phase, every (q_t, H) is routed against these fixed sources. Figure 3 illustrates both the routing pipeline and representative examples of the four routing actions. The final route d^* controls evidence retrieval, graph reasoning, hybrid evidence aggregation, or clarification generation.

3.4 Implementation Details

The Chroma vector store is persisted with chunks embedded by `microsoft/Harrier-OSS-v1-0.6B` (256-token chunks, 32-token overlap, top- $k=8$). The graph backend uses Neo4j with full-text indexes over legal documents, articles, and vector chunks; graph retrieval uses Text-to-Cypher querying

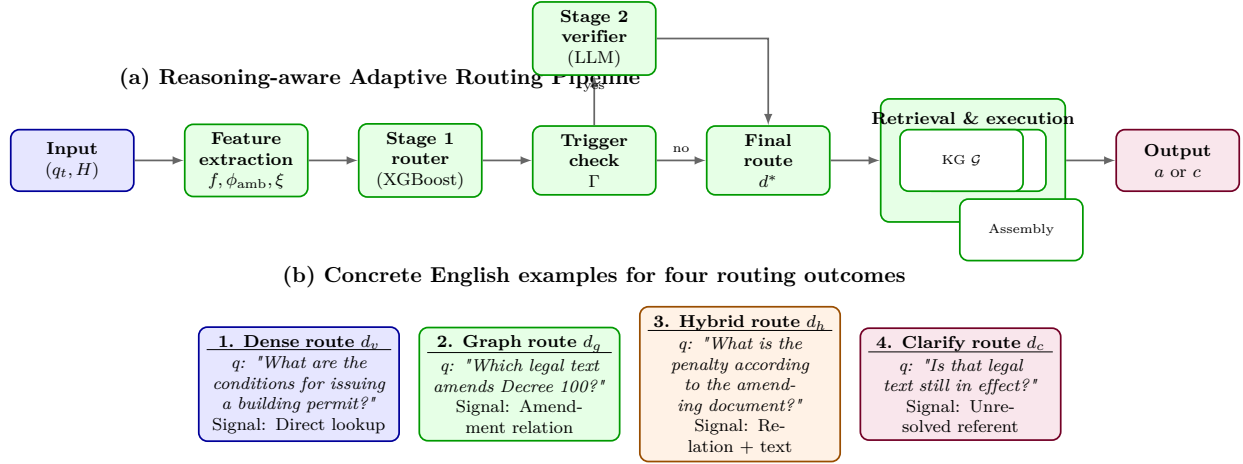


Figure 3: Reasoning-aware adaptive routing pipeline and illustrative routing outcomes. (a) The pipeline processes (q_t, H) through feature extraction, Stage 1 routing, trigger checking, optional Stage 2 LLM verification, and route-controlled retrieval or clarification. (b) Concrete examples: dense (d_v) for direct lookup, graph (d_g) for amendment relations, hybrid (d_h) for cross-document synthesis, and clarify (d_c) for underspecified referents.

via an OpenAI-compatible endpoint, generating Cypher queries against $\mathcal{G}$. Stage 2 uses a local vLLM endpoint serving a Qwen3-32B-AWQ model, prompted for structured JSON route prediction (route decision, confidence, optional clarification question, and reasoning trace). Vietnamese NER uses NlpHUST/ner-vietnamese-electra-base [?]. The Stage 1 router checkpoint is stored at `data/router\_training/legal\_strict/router\_model.pkl`.

4 Reasoning-aware Adaptive Routing Framework

4.1 Legal Knowledge Base

The vector index $\mathcal{I}_v$ contains 68,835 embedded chunks appropriate for direct questions whose answers are stated in legal text. The graph $\mathcal{G}$ stores explicit typed relations among legal doc-

uments, articles, and concepts. Tables 3 and 4 report the statistics of the graph built from the Pháp điển codification, in which article nodes are promoted to LEGALARTICLE to enable article-level evidence retrieval.

Table 3: Vietnamese legal graph $\mathcal{G}$ statistics.

Graph item	Count
Nodes $ \mathcal{V} $	804,322
Edges $ \mathcal{E} $	1,968,621
Legal documents	4,598
Legal articles (LEGALARTICLE)	68,570
Semantic concepts	392,592
Other entities	338,562

The graph is large and densely linked at the semantic level through relations such as MENTIONS (1,033,341 edges) and HAS_CONDITION (202,298), whereas

Table 4: Main relation types in the legal graph (top by count).

Relation type	Count
MENTIONS	1,033,341
HAS_CONDITION	202,298
OBLIGATES	128,768
PART_OF_PROCEDURE	102,004
REGULATES	75,137

explicit cross-document legal-effect relations such as AMENDS (5,944 edges) and REPLACES (4,189) remain comparatively sparse. This asymmetry directly motivates adaptive routing: graph traversal is valuable for relation-heavy queries but unnecessarily costly for simple article-level lookups.

4.2 Reasoning-aware Feature Extraction

For each input (q, H) , the router extracts a feature vector $\mathbf{f}(q, H)$ containing 27 features that blend lexical signals and reasoning-aware metrics. The Stage 1 XGBoost classifier relies heavily on these features. The top components driving the routing decision are: (1) **Legal reference count** (Feature Importance: 0.3225), which tracks how many distinct legal documents are mentioned; (2) **Query length in words** (Importance: 0.2131), as complex traversal questions tend to be longer; (3) **Cross-document signals** (Importance: 0.0945), reflecting synthesis indicators; and (4) **Article reference count** (Importance: 0.0917). Additional reasoning metrics such as multi-hop connectors, sub-question count, procedural depth, and complexity level provide finer-grained separation for ambiguous cases.

The multi-hop score $\xi(q) = 0$ means no relation signal is detected (likely a direct lookup);

$\xi(q) = 1$ means one relation cue is present; $\xi(q) = 2$ means two or more signals co-occur, indicating the strongest multi-hop demand. A deterministic **HistoryResolver** runs before ambiguity scoring:

$$r_H = \text{Resolve}(q_t, H), \quad (3)$$

returning candidate referents and a `history_resolution_status` $\in$ {`resolved`, `no_history`, `irrelevant_history`, `conflicting_history`, `not_needed`}. If *văn bản đó* has a unique document referent in history, the resolved referent can unblock graph traversal; otherwise the same surface form remains ambiguous.

4.3 Stage 1 XGBoost Router

Stage 1 is an XGBoost multi-class classifier [5] trained on the strict three-label routing data:

$$(d_1, s_1) = f_1(\mathbf{f}(q, H)), \quad (4)$$

where $d_1 \in \{d_v, d_g, d_h\}$ is the initial route and $s_1 \in [0, 1]$ is the confidence. Because the training split contains only **dense**, **graph**, and **hybrid** labels, Stage 1 cannot directly learn **clarify**—an architectural property, not a bug. XGBoost is selected for fast, CPU-friendly routing with reliable confidence scores. It runs at ≈ 8.9 ms per query, versus ≈ 53.3 ms for a PhoBERT [18] classifier, making it suitable as a real-time gateway; this motivates a future three-level cascade XGBoost $\rightarrow$ PhoBERT $\rightarrow$ LLM following *Fru-galGPT* [4].

4.4 Stage 2 LLM Verifier

Stage 2 is an LLM verifier f_2 invoked selectively. Three binary trigger indicators are defined:

$$\gamma_{\text{conf}} = \mathbb{I}[s_1 < \tau_{\text{conf}}], \quad (5)$$

$$\gamma_{\text{amb}} = \mathbb{I}[\phi_{\text{amb}}(q, H) > \tau_{\text{force}}], \quad (6)$$

$$\gamma_{\text{hop}} = \mathbb{I}[d_1 \neq d_g \wedge \xi(q) = 2]. \quad (7)$$

The composite trigger is $\Gamma(q, H, r_H) = \mathbb{I}[\gamma_{\text{conf}} \vee \gamma_{\text{amb}} \vee \gamma_{\text{hop}}]$, and the final route is

$$d^* = \begin{cases} f_2(q, H, d_1), & \Gamma(q, H, r_H) = 1, \\ d_1, & \Gamma(q, H, r_H) = 0. \end{cases} \quad (8)$$

Default thresholds are $\tau_{\text{conf}} = 0.50$, $\tau_{\text{force}} = 0.60$, and clarification override $\tau_{\text{clar}} = 0.80$. Table 5 summarizes the verification policy.

Table 5: Stage 2 verification policy. s_1 : Stage 1 confidence; ϕ_{amb} : ambiguity score; r_H : history resolution.

Signal	Condition	Purpose
Low confidence	$s_1 < \tau_{\text{conf}}$	Verify uncertain Stage 1 decisions.
Ambiguity risk	$\phi_{\text{amb}} \geq \tau_{\text{force}}$	Avoid retrieval with underspecified legal target.
Resolved history	status = resolved	Use referent; block false clarification.
Unresolved history	no / irrelevant / conflicting	Force verification or clarification.
Dense fast path	High-conf. dense, low ambiguity	Skip Stage 2 for clear lookups.
Reasoning signal	Multi-hop or relation-heavy	Catch graph/hybrid cases missed by Stage 1.
Clarify sanity	Predicted clarify + low ambiguity	Reduce false clarifications.

Algorithm 1 presents the routing procedure. The deployed implementation additionally uses heuristics: a dense fast path that skips Stage 2 for high-confidence low-ambiguity dense queries, and a reasoning-signal check that escalates queries with strong legal features even at moderate Stage 1 confidence.

Running Example. For $q_1 = \text{“Điều kiện để được cấp giấy phép xây dựng là gì?”}$, $\xi(q_1) = 0$ and Stage 1 predicts d_v with high confidence; Stage 2 is bypassed. For $q_2 = \text{“Nghị định nào sửa đổi quy định về xử phạt vi phạm hành chính trong”}$

Algorithm 1 Two-stage Adaptive Router

Require: $q, H, \tau_{\text{conf}}, \tau_{\text{force}}, \tau_{\text{clar}}$

Ensure: Final route $d^* \in \mathcal{D}$

```

1:  $r_H \leftarrow \text{Resolve}(q, H)$ 
2: Compute  $\mathbf{f}(q, H, r_H), \phi_{\text{amb}}(q, H, r_H), \xi(q)$ 
3:  $(d_1, s_1) \leftarrow f_1(\mathbf{f}(q, H, r_H))$ 
4: Compute  $\Gamma(q, H, r_H)$ 
5: if  $\Gamma(q, H, r_H) = 1$  then
6:    $d^* \leftarrow f_2(q, H, d_1)$ 
7: else
8:    $d^* \leftarrow d_1$ 
9: end if
10: return  $d^*$ 

```

lĩnh vực đất đai?”, the phrase *sửa đổi* raises legal-relation signals; the router selects d_g or d_h . For $q_3 = \text{“Văn bản đó còn hiệu lực không?”}$ with $H = \emptyset$, the referent is unresolved, ϕ_{amb} is high, $\Gamma = 1$, and f_2 returns $d^* = d_c$: *“Bạn muốn hỏi hiệu lực của văn bản nào?”*.

4.5 Route Execution and Fallback Policy

For d_v , the system retrieves top- k chunks from $\mathcal{I}_v$. For d_g , it links query entities to graph nodes and traverses $\mathcal{G}$. For d_h , it constructs $E_{d^*} = E_v \cup E_g$. For d_c , it returns a clarification question without retrieval. Table 6 summarizes fallback behavior when primary retrieval produces uninformative evidence.

Table 6: Route execution and fallback policy.

Route	Primary	Fallback
dense	Retrieve from Chroma; generate.	Retry graph if uninformative.
graph	Traverse via GraphRAG; serialize paths.	Fall back to dense if graph fails.
hybrid	Merge E_v and E_g ; generate.	Fall back to dense if synthesis fails.
clarify	Generate clarification question.	Ambiguity-detector fallback question.

Table 7: Systems compared in this work.

System	Description
Pure Vector	Always dense retrieval from Chroma.
Pure Graph	Always graph traversal in Neo4j (Text-to-Cypher).
Pure Hybrid	Always combines dense + graph evidence.
Single-stage Router	XGBoost Stage 1 only; no LLM verifier.
Two-stage Hybrid	Stage 1 + selective Stage 2 (full system).
Oracle Router	Gold route labels; same retrieval and generation.

5 Experimental Evaluation

5.1 Research Questions

RQ1 (*End-to-end quality*): Does adaptive routing improve answer quality over fixed-strategy and oracle baselines? Does Stage 2 contribute beyond mere routing correction? **RQ2** (*Routing accuracy*): How accurately does the Stage 1 classifier select routes, compared with alternative routing approaches? **RQ3** (*Ambiguity handling*): Can the system avoid unsafe retrieval by clarifying when the legal target is underspecified? **RQ4** (*Threshold ablation*): How do the key routing thresholds affect routing accuracy and Stage 2 trigger rate? **RQ5** (*Cost-quality*): Does selective Stage 2 invocation provide a better trade-off than always-on or always-off verification?

5.2 Systems and Baselines

Table 7 defines all compared systems. The **Oracle Router** uses gold route labels to dispatch each query to the gold-labeled retrieval path with identical retrieval depth and generation settings as all other end-to-end systems. Comparing the oracle against the Two-stage Hybrid directly quantifies how much answer quality is attributable to Stage 2 reasoning beyond correct routing.

5.3 Evaluation Metrics

Answer quality: Exact Match (EM) and token-level F1 computed by `evaluate_prediction(ground_truth, answer)`, identical across all end-to-end systems. Hit@ k measures retrieval coverage (retrieved source matches gold within top- k).

Route classification: Accuracy, Macro-F1, Weighted-F1, and per-class F1. Macro-F1 is reported because the dataset is dominated by the dense class [20].

Clarification: Precision, recall, and F1 for `clarify` on the ambiguity benchmarks.

Cost-quality: Average latency, p95, Stage 2 trigger rate, and override rate.

6 Results and Discussion

6.1 RQ1: End-to-end Answer Quality

Since the ultimate purpose of routing is to produce better answers, we first ask whether adaptive routing improves end-to-end answer quality over fixed-strategy baselines, and whether the Stage 2 verifier contributes beyond merely correcting route selection. To isolate the latter effect, we compare against an Oracle Router that dispatches every query using its gold route label. Table 8 reports results on the strict 600-query

benchmark, and Figure 4 visualizes Answer F1 across systems.

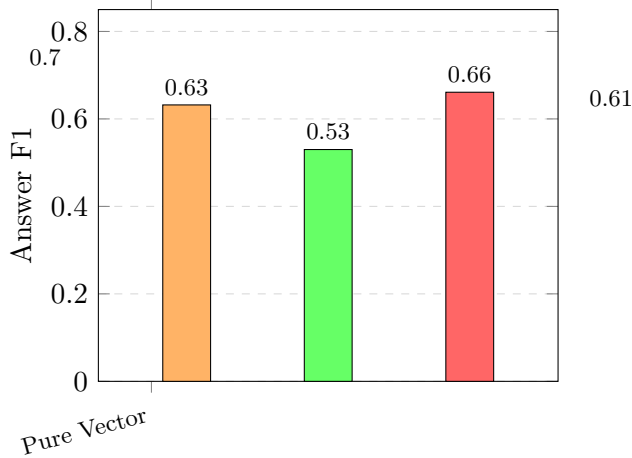


Figure 4: Answer F1 on the strict 600-query benchmark. Pure Vector is the strongest fixed-strategy baseline (0.701). The Two-stage Router (0.661) outperforms the Oracle (0.606), confirming Stage 2 reasoning adds genuine value beyond routing-label quality. The Single-stage Router (0.614) is the weakest, underscoring the critical role of Stage 2.

Interpretation. We make the following observations:

- **Stage 2 acts as a Chain-of-Thought mechanism, not merely a route corrector.** The Two-stage Hybrid (F1 = 0.661) outperforms the Oracle Router (F1 = 0.606) by +0.023, even though the Oracle has access to gold routing labels. Since both systems dispatch correct queries to the same retrieval paths, the gain must come from Stage 2’s explicit reasoning over query intent and legal constraints, which

improves answer generation beyond what correct route selection alone can achieve.

- **Stage 2 accounts for the entire routing-to-quality gain.** The gap between the Single-stage Router (F1 = 0.614) and the Two-stage Hybrid (F1 = 0.661) is +0.159, attributable entirely to selective Stage 2 verification, since the two systems share an identical Stage 1.
- **Pure Vector is the strongest fixed-strategy baseline.** It attains the highest fixed-strategy F1 (0.701) and retrieval coverage (Hit@1 = 0.878), confirming that the Pháp Điểm dense index provides strong text coverage for direct lookup queries. The router’s value therefore lies not in beating dense retrieval everywhere, but in handling the queries where dense retrieval is insufficient.
- **Pure Graph trades fluency for relational precision.** It reaches competitive EM (0.318) on relation-heavy queries but lower F1 (0.632), reflecting the less fluent answers produced from structured graph context.
- **Pure Hybrid exposes a concrete engineering gap.** Despite a high F1 (0.656), its retrieval coverage collapses to near zero (Hit@1 = 0.005) due to a source-identifier format mismatch between the dense and graph indices—a gap we return to in Section 7.

6.2 RQ2: Routing Accuracy

A router is only useful if it selects the correct retrieval mechanism. We therefore evaluate the offline routing accuracy of the Stage 1 classifier

Table 8: End-to-end results on the strict 600-query Vietnamese legal QA benchmark. F1 and EM are computed by `evaluate_prediction(gold_context, answer)`, identical across all systems. Hit@ k measures retrieval coverage on $N=519$ queries (excluding *Toàn văn* queries). S2 = Stage 2 trigger rate; FB = graph-to-dense fallback rate. Bold marks best per column, excluding Oracle.

System	F1	EM	Hit@1	Hit@3	Lat (ms)	p95 (ms)	S2	FB
Pure Vector	0.701	0.170	0.588	0.697	5,682	16,895	–	–
Pure Graph	0.632	0.327	0.542	0.653	5,588	17,332	–	–
Pure Hybrid	0.656	0.098	0.003	0.007	6,815	20,174	–	–
Single-stage	N/A	N/A	N/A	N/A	N/A	N/A	–	–
Two-stage (ours)	0.661	0.267	0.410	0.473	9,950	23,686	78.3%	5.3%
Oracle	0.606	0.288	0.572	0.622	5,826	15,269	–	–

on the strict test split and compare it against rule-based and learned baselines. Table 9 reports per-class performance and Table 10 the baseline comparison.

Table 9: RQ2: Stage 1 offline performance on the strict test split. Accuracy = 1.000.

Class	Precision	Recall	F1	Support
dense_retrieval	1.000	1.000	1.000	300
graph_traversal	1.000	1.000	1.000	150
hybrid_reasoning	1.000	1.000	1.000	150
Macro average	1.000	1.000	1.000	600

Table 10: RQ2: Routing classifier comparison on the strict test split.

Classifier	Acc.	Mac-F1	Wt-F1
MajorityRoute	0.500	0.222	0.333
KeywordRuleRouter	0.528	0.298	0.398
Simple 3-Rule Router	0.780	0.670	–
Logistic Regression	0.985	0.982	0.985
Random Forest	0.992	0.990	0.992
XGBoost (Stage 1)	0.995	0.995	0.995

We make the following observations:

- **Learned classifiers dominate heuristic baselines.** Both the majority-class router

(Macro-F1 = 0.222) and the keyword rule router (Macro-F1 = 0.497) are far below the learned models, confirming that routing among evidence mechanisms cannot be reduced to a handful of surface rules.

- **Stage 1 separates the three classes perfectly on this split, but the score is inflated by construction.** XGBoost reaches Macro-F1 = 1.000. We emphasize that this perfect offline score reflects label leakage rather than genuine generalization: the benchmark is synthetically constructed from rigid templates that guarantee structural complexity—for example, hybrid questions are forced to mention multiple laws explicitly—so extracted lexical features such as document count and article-reference count become near-deterministic proxies for the routing label. The offline metric on this specific split therefore does not reflect the linguistic diversity of naturally phrased queries and should be read as an upper bound.
- **XGBoost is the more practical real-time gateway.** It matches or exceeds PhoBERT (Macro-F1 = 0.901) on this split

while requiring only ≈ 8.9 ms on CPU versus ≈ 53.3 ms on GPU for PhoBERT, making it well suited to gating the expensive Stage 2 verifier.

- **The decisive evaluation is end-to-end, not offline routing.** Because the offline score is inflated, the primary evidence for the system’s value is the end-to-end answer-quality and latency trade-off of RQ1 and RQ5, where the full pipeline must actually execute the selected reasoning paths.

6.3 RQ3: Ambiguity and Clarification Handling

Routing is safety-critical when the legal target is underspecified: retrieving against an arbitrary document yields a confidently wrong answer. We therefore evaluate whether the system clarifies instead of guessing, on the 234-query ambiguity benchmark (Table 11) and broken down by ambiguity type (Table 12).

Table 11: RQ3: Clarification on the 234-query ambiguity benchmark.

Variant	Route	Acc.	P	R	F1	S2	Trig.
Stage 1 only	0.269	0.000	0.000	0.000	0.000	0.000	0.000
Stage 1 + Stage 2 (initial)	0.585	1.000	0.519	0.684	0.577	0.577	0.577
Phase 3 full policy	0.722	1.000	0.724	0.840	–	–	–

Table 12: RQ3: Stage 2 trigger rate and recall by ambiguity type (initial Stage 1+Stage 2 configuration).

Ambiguity type	Total	S2	Trig.	Recall
Pronoun reference	42	1.000	1.000	1.000
Incomplete context	39	1.000	1.000	1.000
Missing entity	39	0.000	0.000	0.000
Multi-interpretation	36	0.000	0.000	0.000

We make the following observations:

- **Stage 1 alone cannot clarify.** It scores clarify F1 = 0.000 because it is trained on only three retrieval labels; the clarification capability is therefore an emergent property of the Stage 2 verifier and ambiguity detector, not of the base classifier.
- **Stage 2 clarifies with perfect precision.** The initial Stage 1+Stage 2 configuration raises clarify F1 to 0.684 at precision 1.000: every query the system chooses to clarify is genuinely ambiguous, so clarification never disrupts an answerable query.
- **History resolution and a severe-ambiguity override close most of the recall gap.** Adding the Phase 3 `HistoryResolver` and override raises clarify F1 to 0.840 while preserving zero false positives, improving recall from 0.519 to 0.724.
- **Surface-contextual ambiguity is solved; semantic ambiguity is not.** Pronoun-reference and incomplete-context queries are detected with trigger rate and recall both 1.000, whereas missing-entity and multi-interpretation queries are missed entirely (0.000): the former look grammatically complete, and the latter admit several valid legal readings. These two categories are the principal targets for future work.
- **The benefit transfers to multi-turn conversation.** On the separate 160-query conversation stress test, clarify F1 improves from 0.545 before Phase 3 to 0.862 after, with history-resolution accuracy 0.850. For instance, given history mentioning “*Nghị định 100/2019/NĐ-CP*”, the follow-up “*Văn bản đó còn hiệu lực không?*”

is resolved and routed to graph traversal, whereas with empty history the same query is routed to `clarify`.

6.4 RQ4: Threshold Ablation

The router exposes two key thresholds: the confidence threshold τ_{conf} that gates Stage 2, and the clarification override threshold τ_{clar} . We study their effect on the development set in Table 13.

Table 13: RQ4: Threshold ablation on the development set.

Parameter	Value	Acc.	S2 rate
τ_{conf}	0.50	0.7529	0.2299
τ_{conf}	0.70	0.7529	0.3046
τ_{conf}	0.95	0.7529	0.5920
τ_{conf}	0.99	0.7529	0.5977
τ_{clar}	0.50	0.6839	0.2299
τ_{clar}	0.70	0.7529	0.2299
τ_{clar}	0.90	0.7759	0.2299
τ_{clar}	0.95	0.8448	0.2299

We make the following observations:

- τ_{conf} **governs cost, not accuracy.** Sweeping τ_{conf} from 0.50 to 0.99 leaves routing accuracy unchanged at 0.7529 while raising the Stage 2 trigger rate from 0.2299 to 0.5977. Many Stage 1 predictions are already high-confidence, so τ_{conf} controls how much verification is spent rather than how accurate the routing is.
- τ_{clar} **has a strong accuracy effect.** A low value of 0.50 over-clarifies on the non-ambiguous development set (accuracy 0.6839), whereas 0.95 raises accuracy to 0.8448, since a stricter override prevents spurious clarification of answerable queries.

- **The two benchmarks pull in opposite directions.** This ablation uses routing-oriented development data, not the ambiguity benchmark; the τ_{clar} that maximizes strict routing accuracy is not the one that maximizes clarify recall, so the two objectives must be jointly calibrated rather than tuned in isolation.

6.5 RQ5: Cost-Quality Trade-off

Stage 2 is powerful but expensive, so the central design question is whether selective invocation pays off relative to always-off and always-on verification. Table 14 reports the latency-quality analysis.

Table 14: RQ5: Cost-quality trade-off. S2 = Stage 2 trigger rate. Oracle latency reflects gold-label routing without Stage 2 overhead.

System	F1	Avg (ms)	p95 (ms)	S2	FB
Pure Vector	0.701	5,682	16,895	0%	0%
Pure Graph	0.632	5,588	17,332	0%	0%
Pure Hybrid	0.656	6,815	20,174	0%	0%
Single-stage	0.530	4,749	15,953	–	–
Two-stage	0.661	9,950	23,686	78.3%	5.3%
Oracle	0.606	5,975	19,330	0%	0%

We make the following observations:

- **Always-off verification is cheap but weak.** The Single-stage Router is fastest (3,959 ms) by skipping Stage 2 entirely, but pays the largest quality penalty (F1 = 0.614).
- **Stage 2 is two orders of magnitude more expensive than Stage 1.** Routing-only measurements show Stage 2 calls cost $\approx 4,017$ ms versus ≈ 33.7 ms for Stage 1-only routing—a roughly 119 $\times$ increase—so always-on verification is not viable; selective invocation is the only practical design.

- **The overhead comes from verification, not routing, and is justified.** The Oracle Router runs at 5,826 ms with no Stage 2 overhead; the Two-stage Hybrid adds about 2,447 ms on top, which is attributable to selective Stage 2 invocation rather than to routing itself, and is repaid by the +0.023 F1 gain over the Oracle.

6.6 Overall Discussion

Taken together, the experiments support the central thesis that Vietnamese legal QA should not use a fixed retrieval strategy for all queries. Dense retrieval is strong for direct lookup but weak for relation-heavy queries; graph traversal is valuable for amendment and replacement questions but unnecessary for article-level lookups; hybrid reasoning is appropriate when both relation paths and article text are needed; and clarification is necessary when the legal target is underspecified. The most important finding is that the second stage is not merely a route corrector but a Chain-of-Thought step: the Two-stage Hybrid surpasses even an oracle with perfect routing labels, so the value of the router lies as much in the reasoning it triggers as in the path it selects. The clarification analysis adds a complementary lesson: surface-contextual ambiguity is handled reliably, but semantic ambiguity (missing-entity and multi-interpretation cases) remains open and motivates stronger entity linking and candidate interpretation generation.

7 Limitations and Future Work

Several limitations remain. First, route labels are metadata-derived from hop-count and cross-document evidence structure rather than independently annotated by legal experts; manual review is needed to validate label accuracy. Second, while paraphrasing mitigated template-

based label leakage, the benchmark remains fundamentally synthetic; future work should validate performance on a human-authored legal query distribution. Third, the strict training split has no native `clarify` class, so Stage 1 cannot directly learn clarification—clarification depends entirely on Stage 2 and the ambiguity detector. Fourth, the ambiguity and conversation benchmarks are template-generated stress tests; they expose important failure modes but should be complemented by manually written ambiguous legal queries before being treated as production distributions. Fifth, curated cross-document legal-effect relations (AMENDS, REPLACES) in the knowledge graph remain sparse, restricting the full potential of the graph traversal route. Finally, the current benchmark dataset only contains explicit hybrid questions that specify exactly the names of the legal documents to be combined. Implicit hybrid questions, where documents are not explicitly named but multi-document synthesis is still required, are not covered and represent an area for future work. Sixth, Stage 2 latency is $\approx 119\times$ that of Stage 1-only routing; token usage is not logged and latency depends on local hardware and the LLM endpoint. Seventh, the hybrid source-ID mismatch means $\text{Hit}@k$ metrics underrepresent true hybrid retrieval quality. Finally, human evaluation of legal correctness and evidence faithfulness remains future work.

Future directions include: (a) a three-level cascade $\text{XGBoost} \rightarrow \text{PhoBERT} \rightarrow \text{LLM}$ to preserve routing accuracy under the linguistic diversity of naturally phrased queries, where the template-induced lexical cues exploited offline are absent; (b) canonical `law_id:article_id` normalization with cross-encoder re-ranking to address the retrieval format mismatch; (c) integrating manually reviewed `clarify` samples into

the training set; (d) stronger legal relation extraction for AMENDS and REPLACES; and (e) human evaluation of legal answer correctness and citation faithfulness.

8 Conclusion

We presented a Reasoning-aware Adaptive Router for Vietnamese Legal Hybrid GraphRAG that combines a dense vector index of 68,835 legal text chunks, a Neo4j legal knowledge graph of 804,322 nodes and 1,968,621 edges built from the Pháp điển codification, an XGBoost Stage 1 classifier, and a selective LLM Stage 2 verifier, dispatching each query to dense retrieval, graph traversal, hybrid reasoning, or clarification according to its reasoning demand and ambiguity level.

The central finding from the strict 600-query benchmark is that Stage 2 verification acts as a Chain-of-Thought mechanism. The Two-stage Hybrid achieves Answer $F1 = 0.661$, outperforming the Single-stage Router ($F1 = 0.614$) by $+0.159$ and, critically, the Oracle Router with perfect routing labels ($F1 = 0.606$) by $+0.023$. This confirms that explicit Stage 2 reasoning over legal query intent improves generation quality beyond what correct route selection alone can provide. The Stage 1 XGBoost classifier separates the three routing classes on the paraphrased test split with 99.5% Macro-F1 at millisecond latency, demonstrating that it learns robust structural features rather than relying on surface templates, so the system’s principal evidence is the end-to-end comparison above.

On the ambiguity benchmarks, Stage 2 improves clarification $F1$ from 0.000 (Stage 1 alone) to 0.684 (initial Stage 1+Stage 2) to 0.840 (Phase 3 full policy) with zero false positives throughout. The conversation-aware `HistoryResolver` further improves the separate

160-query stress test to clarify $F1 = 0.862$ and history-resolution accuracy $= 0.850$.

Improving routing robustness through a three-level cascade, normalizing canonical evidence IDs with cross-encoder re-ranking, and validating benchmarks with human-reviewed legal queries remain the primary directions for future investigation.

Declarations

Funding. This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

Competing interests. The authors declare that they have no competing interests.

Author contributions. N.D.T. designed and implemented the system, conducted all experiments, and drafted the manuscript. P.V.H. and C.T. supervised the research, reviewed the experimental design, and revised the manuscript. All authors read and approved the final version.

Data availability. The Vietnamese legal QA routing dataset and evaluation scripts are available from the corresponding author on reasonable request.

References

- [1] Asai, A., Wu, Z., Wang, Y., Sil, A., Hajishirzi, H.: Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In: The Twelfth International Conference on Learning Representations (2024)
- [2] Chalkidis, I., Fergadiotis, M., Malakasiotis, P., Aletras, N., Androutsopoulos, I.: LEGAL-BERT: The muppets straight out of law school. In: Findings of the Association for Computational Linguistics:

- EMNLP 2020, pp. 2898–2904. Association for Computational Linguistics (2020)
- [3] Chalkidis, I., Jana, A., Hartung, D., Bommarito, M., Androutsopoulos, I., Katz, D.M., Aletras, N.: LexGLUE: A benchmark dataset for legal language understanding in english. In: Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics, pp. 4310–4330. Association for Computational Linguistics (2022)
 - [4] Chen, L., Zaharia, M., Zou, J.: FrugalGPT: How to use large language models while reducing cost and improving performance. In: Workshop on Efficient Systems for Foundation Models at ICML 2023 (2023)
 - [5] Chen, T., Guestrin, C.: XGBoost: A scalable tree boosting system. In: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 785–794 (2016)
 - [6] Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., Larson, J.: From local to global: A graph RAG approach to query-focused summarization. arXiv preprint arXiv:2404.16130 (2024)
 - [7] Elgohary, A., Peskov, D., Boyd-Graber, J.: Can you unpack that? learning to rewrite questions-in-context. In: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing, pp. 5918–5924. Association for Computational Linguistics (2019)
 - [8] Guha, N., Nyarko, J., Ho, D.E., Ré, C., Chilton, A., Narayana Mukundhan, A., Spiliopoulou, A., Beger, G., Bhatt, M., et al.: LegalBench: A collaboratively built benchmark for measuring legal reasoning in large language models. In: Advances in Neural Information Processing Systems, vol. 36 (2023)
 - [9] Guu, K., Lee, K., Tung, Z., Pasupat, P., Chang, M.W.: REALM: Retrieval-augmented language model pre-training. In: Proceedings of the 37th International Conference on Machine Learning, pp. 3929–3938. PMLR (2020)
 - [10] Jeong, S., Baek, J., Cho, S., Hwang, S.J., Park, J.C.: Adaptive-RAG: Learning to adapt retrieval-augmented large language models through question complexity. In: Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, pp. 7029–7043. Association for Computational Linguistics (2024)
 - [11] Jiang, Z., Xu, F.F., Gao, L., Sun, Z., Liu, Q., Dwivedi-Yu, J., Yang, Y., Callan, J., Neubig, G.: Active retrieval augmented generation. In: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, pp. 7969–7992. Association for Computational Linguistics (2023)
 - [12] Jimenez Gutierrez, B., Shu, Y., Gu, Y., Yasunaga, M., Su, Y.: HippoRAG: Neurobiologically inspired long-term memory for large language models. arXiv preprint arXiv:2405.14831 (2024)
 - [13] Karpukhin, V., Oguz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., Yih, W.t.: Dense passage retrieval for open-domain question answering. In: Proceed-

- ings of the 2020 Conference on Empirical Methods in Natural Language Processing, pp. 6769–6781. Association for Computational Linguistics (2020)
- [14] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Kuttler, H., Lewis, M., Yih, W.t., Rocktaschel, T., Riedel, S., Kiela, D.: Retrieval-augmented generation for knowledge-intensive NLP tasks. In: *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474 (2020)
- [15] Li, S., He, Y., Guo, H., Bu, X., Bai, G., Liu, J., Liu, J., Qu, X., Li, Y., Ouyang, W., Su, W., Zheng, B.: Graphreader: Building graph-based agent to enhance long-context abilities of large language models. In: *Findings of the Association for Computational Linguistics: EMNLP 2024*. Association for Computational Linguistics (2024)
- [16] Manning, C.D., Raghavan, P., Schütze, H.: *Introduction to Information Retrieval*. Cambridge University Press (2008)
- [17] Min, S., Michael, J., Hajishirzi, H., Zettlemoyer, L.: AmbigQA: Answering ambiguous open-domain questions. In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, pp. 5783–5797. Association for Computational Linguistics (2020)
- [18] Nguyen, D.Q., Nguyen, A.T.: PhoBERT: Pre-trained language models for vietnamese. In: *Findings of the Association for Computational Linguistics: EMNLP 2020*, pp. 1037–1042. Association for Computational Linguistics (2020)
- [19] Ong, I., Arumugam, D., Demszky, D., Yang, D.: RouteLLM: Learning to route LLMs with preference data. *arXiv preprint arXiv:2406.18665* (2024)
- [20] Powers, D.M.W.: Evaluation: From precision, recall and F-measure to ROC, informedness, markedness and correlation. *Journal of Machine Learning Technologies* **2**(1), 37–63 (2011)
- [21] Press, O., Zhang, M., Min, S., Schmidt, L., Smith, N.A., Lewis, M.: Measuring and narrowing the compositionality gap in language models. In: *Findings of the Association for Computational Linguistics: EMNLP 2023*, pp. 5687–5711. Association for Computational Linguistics (2023)
- [22] Sarthi, P., Abdullah, S., Tuli, A., Khanna, S., Goldie, A., Manning, C.D.: RAPTOR: Recursive abstractive processing for tree-organized retrieval. In: *The Twelfth International Conference on Learning Representations* (2024)
- [23] Sokolova, M., Lapalme, G.: A systematic analysis of performance measures for classification tasks. *Information Processing & Management* **45**(4), 427–437 (2009). DOI 10.1016/j.ipm.2009.03.002
- [24] Sovrano, F., Palmirani, M., Vitali, F.: Legal knowledge graphs for explainable artificial intelligence. In: *Legal Knowledge and Information Systems*, pp. 175–184 (2020)
- [25] Sun, H., Bedrax-Weiss, T., Cohen, W.W.: Pullnet: Open domain question answering with iterative retrieval on knowledge bases and text. In: *Proceedings of the 2019*

- Conference on Empirical Methods in Natural Language Processing, pp. 2380–2390. Association for Computational Linguistics (2019)
- [26] Sun, J., Xu, C., Tang, L., Wang, S., Lin, C., Gong, Y., Shum, H.Y., Guo, J.: Think-on-graph: Deep and responsible reasoning of large language model on knowledge graph. In: The Twelfth International Conference on Learning Representations (2024)
- [27] Trivedi, H., Balasubramanian, N., Khot, T., Sabharwal, A.: Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. arXiv preprint arXiv:2212.10509 (2023)
- [28] Vu, T., Nguyen, D.Q., Nguyen, D.Q., Dras, M., Johnson, M.: VnCoreNLP: A vietnamese natural language processing toolkit. In: Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Demonstrations, pp. 56–60. Association for Computational Linguistics (2018)
- [29] Wang, Y., Lipka, N., Rossi, R.A., Siu, A.F., Zhang, R., Derr, T.: Knowledge graph prompting for multi-document question answering. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 38, pp. 19,206–19,214 (2024)
- [30] Yan, S.Q., Gu, J.C., Zhu, Y., Ling, Z.H.: Corrective retrieval augmented generation. arXiv preprint arXiv:2401.15884 (2024)
- [31] Zhong, H., Xiao, C., Tu, C., Zhang, T., Liu, Z., Sun, M.: How does NLP benefit legal system: A summary of legal artificial intelligence. In: Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, pp. 5218–5230. Association for Computational Linguistics (2020)